

实验进阶2 银行业务数据录入与修改

难度（最高三星）：★★★ 建议学时（2-4学时）：2学时

实验说明

学习地图

某银行是一家小型银行企业，有十多万客户。现在要为该银行开发一套银行业务系统，通过计算机对银行日常的业务进行处理，以保证数据的安全性，提高工作效率。

本系统的功能结构图如图1所示。

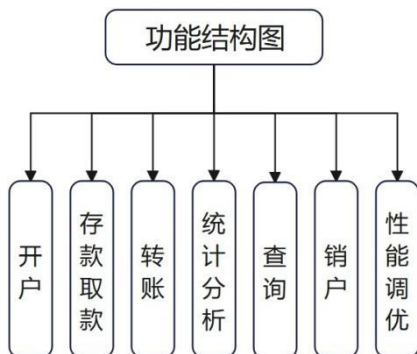


图1 银行业务系统功能结构图

本项目实验案例集要求根据银行业务需求设计出符合第3范式的数据库结构，使用SQL语言创建数据库和表，并添加表约束，进行数据的增、删、改、查，运用触发器、事务、视图和存储过程，按照银行的业务需求实现各项业务（包括日常存款、取款和转账等），同时为了适应日益增长的客户需求，要对数据库进行性能调优。

本项目将完成一个典型的数据库管理系统——银行业务系统的项目开发，要求完成的项目能够满足用户需求中的功能要求。共分为5个实验进阶、15个任务，按照难度等级分为初级、中级、高级，如图2所示。



图2 银行业务系统数据库

整个项目实验案例集将涉及openGauss数据库多个知识点和技能，如图3所示。

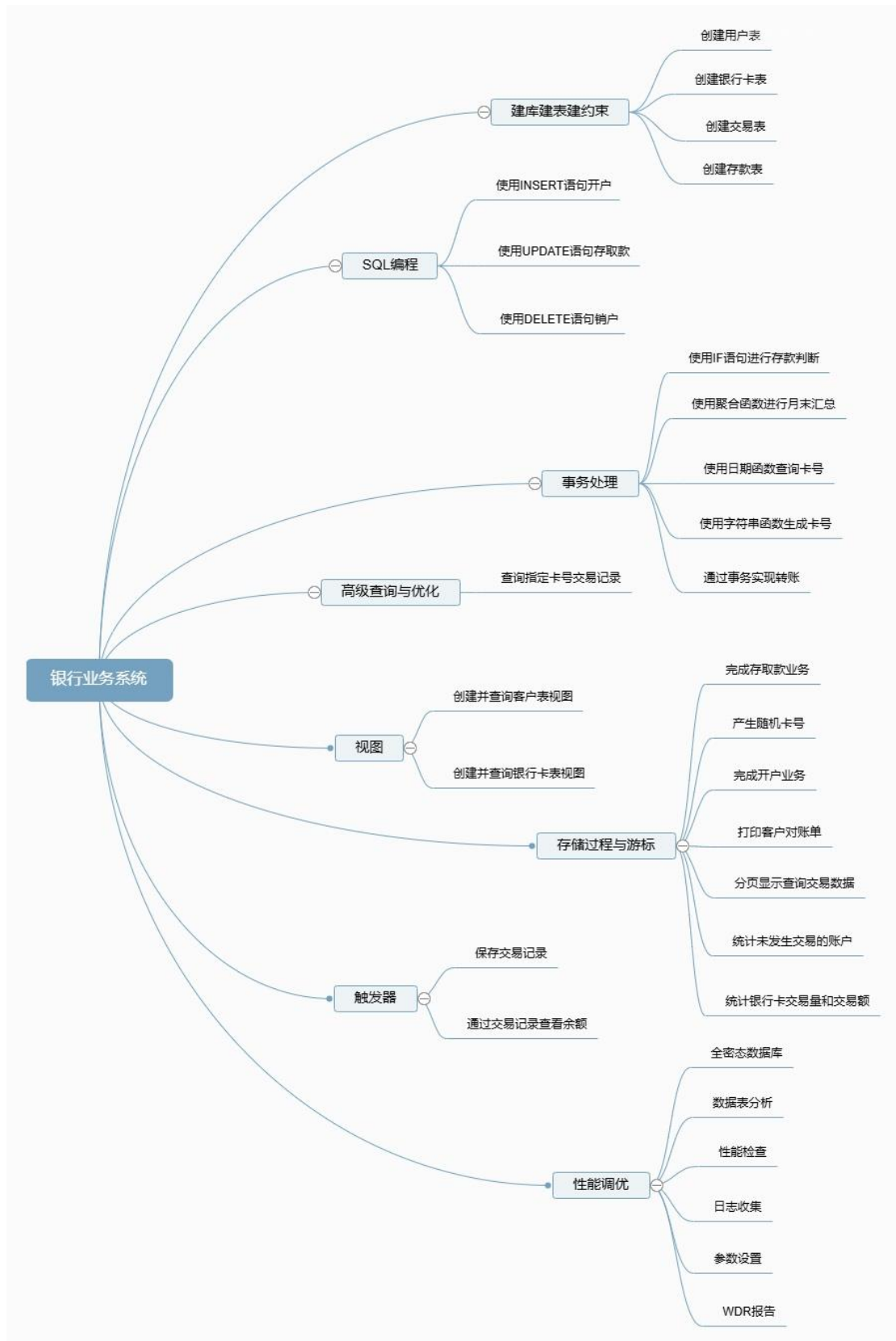


图3 银行业务系统知识点和技能

实验环境准备

具体开发环境如下：

- (1) 数据库：openGauss 5.0.2
- (2) 可视化开发工具：Data Studio
- (3) 终端工具：FinalShell，其安装和建立连接步骤见[网址](#)。
- (4) 数据库创建：该部分请参照实验进阶1【[银行业务系统数据库表设计](#)】中任务完成，

或直接[下载相关代码](#)（如下）运行。

```
CREATE DATABASE bankdb;

CREATE USER bank_admin WITH PASSWORD'Bank@1234';

GRANT ALL PRIVILEGES ON DATABASE bankdb TO bank_admin;


CREATE SCHEMA bank_schema;
SET SEARCH_PATH TO bank_schema


CREATE TABLE userInfo
(
    customerID SERIAL PRIMARY KEY,
    customerName CHAR(8) NOT NULL,
    PID CHAR(18) NOT NULL,
    telephone CHAR(20) NOT NULL,
    address VARCHAR(50)
);


CREATE TABLE cardInfo
(
    cardID CHAR(17) PRIMARY KEY,
    curlID VARCHAR(10) DEFAULT 'RMB' NOT NULL,
    savingID INT NOT NULL DEFAULT 1,
    openDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
openMoney NUMERIC(18,2) NOT NULL,
balance NUMERIC(18,2) NOT NULL,
pass CHAR(6) NOT NULL DEFAULT '888888',
IsReportLoss VARCHAR(3) NOT NULL DEFAULT 'F',
customerID INT NOT NULL
);

CREATE TABLE tradeInfo
(
tradeDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
tradeType CHAR(6) NOT NULL,
cardID CHAR(19) NOT NULL,
tradeMoney NUMERIC(18,2) NOT NULL,
remark TEXT
);

CREATE TABLE deposit
(
savingID SERIAL PRIMARY KEY,
savingName VARCHAR(20) NOT NULL,
descrip VARCHAR(50)
);

ALTER TABLE userInfo
ADD CONSTRAINT chk_PID CHECK (CHAR_LENGTH(CAST(PID AS TEXT)) = 18),
ADD CONSTRAINT chk_telephone CHECK
(CHAR_LENGTH(REGEXP_REPLACE(CAST(telephone AS TEXT), '[^0-9]', '', 'g')) = 11);

ALTER TABLE cardInfo
ADD CONSTRAINT ck_cardid CHECK (
SUBSTRING(cardID, 1, 8) = '10103576'
AND SUBSTRING(cardID, 9) ~ '^ [0-9]{8}$'
),
ADD CONSTRAINT CK_openMoney CHECK (openMoney >= 1),
```

```
ADD CONSTRAINT CK_balance CHECK (balance >=1),  
ADD CONSTRAINT CK_pass CHECK (LENGTH(pass) = 6AND pass ~ '^[0-9]+$'),  
ADD CONSTRAINT FK_customerID FOREIGN KEY(customerID) REFERENCES  
userInfo(customerID),  
ADD CONSTRAINT FK_savingID FOREIGN KEY(savingID) REFERENCES deposit(savingID);  
  
ALTER TABLE tradeInfo  
    ADD CONSTRAINT CK_tradeType CHECK(tradeType IN ('Credit', 'Debit')),  
    ADD CONSTRAINT CK_tradeMoney CHECK(tradeMoney > 0);
```

实验任务

任务描述

本次实验进阶2，主要实现对银行业务数据的录入与修改、以及简单业务实现，共计三个任务，包括：

- 任务1：录入数据；
- 任务2：修改密码；
- 任务3：挂失银行卡。

学习目标

通过这三项任务能够快速掌握openGauss的SQL语法知识，能够完成数据的增删改查。

任务准备

前置知识

本实验案例，需提前学习openGauss的基本概念、作用及基础操作。可通过如下途径进行学习：

- 数字教材：《[数据库原理与应用：基于openGauss](#)》
- 技术文档：[openGauss白皮书](#)

任务实施

本实验的具体操作可参见视频。

任务 1：录入数据

向存款类型表 (deposit)插入存款类型数据。插入数据的示例如下。

```
INSERT INTO deposit (savingName,descrip) VALUES ('活期','按存款日结算利息');
INSERT INTO deposit (savingName,descrip) VALUES ('定期一年','存款期是1年');
INSERT INTO deposit (savingName,descrip) VALUES ('定期二年','存款期是2年');
INSERT INTO deposit (savingName,descrip) VALUES ('定期三年','存款期是3年');
INSERT INTO deposit (savingName) VALUES ('定活两便');
INSERT INTO deposit (savingName,descrip) VALUES ('零存整取一年','存款期是1年');
INSERT INTO deposit (savingName,descrip) VALUES ('零存整取二年','存款期是2年');
INSERT INTO deposit (savingName,descrip) VALUES ('零存整取三年','存款期是3年');
INSERT INTO deposit (savingName,descrip) VALUES ('存本取息五年','按月支取利息');
```

向用户信息表 (userInfo) 和银行卡信息表 (cardInfo)插入用户信息和银行卡信息。

插入信息的示例如下。

```
INSERT INTO userInfo(customerid,customerName,PID,telephone,address)
```

```
VALUES(1,'John','11000000000000000001','13900000001','江苏省南京市');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(2,'Jane','11000000000000000002','13900000002');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(3,'Michael','11000000000000000003','13900000003');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(4,'Emily','11000000000000000004','13900000004');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(5,'David','11000000000000000005','13900000005');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(6,'Sarah','11000000000000000006','13900000006');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(7,'Robert','11000000000000000007','13900000007');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(8,'Jessica','11000000000000000008','13900000008');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(9,'William','11000000000000000009','13900000009');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(10,'Olivia','11000000000000000010','13900000010');
```

```
INSERT INTO
cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000001','RMB',1,1000.00,1000.00,'888888','否',1);
INSERT INTO
cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000002','RMB',2,1.00,1.00,'888888','否',2);
INSERT INTO
cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000003','RMB',2,1.00,1.00,'888888','否',3);
INSERT INTO
cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000004','RMB',2,1.00,1.00,'888888','否',4);
```



```

INSERT INTO
cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000005','RMB',3,500.00,500.00,'888888','否',5);

INSERT INTO
cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000006','RMB',3,300.00,300.00,'888888','否',6);

INSERT INTO
cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000007','RMB',3,200.00,200.00,'888888','否',7);

INSERT INTO
cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000008','RMB',3,150.00,150.00,'888888','否',8);

INSERT INTO
cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000009','RMB',3,100.00,100.00,'888888','否',9);

INSERT INTO
cardInfo(cardid,curid,savingid,openmoney,balance,pass,isreportloss,customerid)
VALUES('1010357600000010','RMB',3,80.00,80.00,'888888','否',10);

```

插入完成后用查询语句查看各个表的情况，如图1.1-图1.3所示。

```
SELECT * FROM deposit;
```

	savingid	savingname	descrip
1	1	活期	按存款日结算利息
2	2	定期一年	存款期是1年
3	3	定期二年	存款期是2年
4	4	定期三年	存款期是3年
5	5	定活两便	[NULL]
6	6	零存整取一年	存款期是1年
7	7	零存整取二年	存款期是2年
8	8	零存整取三年	存款期是3年
9	9	存本取息五年	按月支取利息

图1.1 存款类型表

```
SELECT * FROM userinfo;
```

	customerid	customername	pid	telephone	address
1	1	John	110000000000000001	13900000001	江苏省南京市
2	2	Jane	110000000000000002	13900000002	[NULL]
3	3	Michael	110000000000000003	13900000003	[NULL]
4	4	Emily	110000000000000004	13900000004	[NULL]
5	5	David	110000000000000005	13900000005	[NULL]
6	6	Sarah	110000000000000006	13900000006	[NULL]
7	7	Robert	110000000000000007	13900000007	[NULL]
8	8	Jessica	110000000000000008	13900000008	[NULL]
9	9	William	110000000000000009	13900000009	[NULL]
10	10	Olivia	110000000000000010	13900000010	[NULL]

图1.2 用户信息表

```
SELECT * FROM cardinfo;
```

	cardid	curid	savingid	opendate	openmoney	balance	pass	isreportloss	customerid
1	1010357600000001	RMB	1	2024-08-08 15:34:09	1000.00	1000.00	888888	否	1
2	1010357600000002	RMB	2	2024-08-08 15:40:08	1.00	1.00	888888	否	2
3	1010357600000003	RMB	2	2024-08-08 15:40:15	1.00	1.00	888888	否	3
4	1010357600000004	RMB	2	2024-08-08 15:40:21	1.00	1.00	888888	否	4
5	1010357600000005	RMB	3	2024-08-08 15:40:27	500.00	500.00	888888	否	5
6	1010357600000006	RMB	3	2024-08-08 15:40:33	300.00	300.00	888888	否	6
7	1010357600000007	RMB	3	2024-08-08 15:40:39	200.00	200.00	888888	否	7
8	1010357600000008	RMB	3	2024-08-08 15:40:45	150.00	150.00	888888	否	8
9	1010357600000009	RMB	3	2024-08-08 15:41:53	100.00	100.00	888888	否	9
10	1010357600000010	RMB	3	2024-08-08 15:41:58	80.00	80.00	888888	否	10

图1.3 银行卡信息表

任务 2：修改密码

修改客户密码要求如下。

修改Olivia（卡号为1010357600000010）银行卡密码为123456。

修改Emily（卡号为1010357600000004）银行卡密码为123123。

```
UPDATE cardinfo
SET pass = '123456'
WHERE cardid = '1010357600000010';

UPDATE cardinfo
SET pass = '123123'
WHERE cardid = '1010357600000004';
```

9	1010357600000010	RMB	3	2025-03-03 11:38:24	80.00	80.00	123456	否
10	1010357600000004	RMB	2	2025-03-03 11:38:24	1.00	1.00	123123	否

图1.4 修改密码

任务 3：挂失银行卡

Robert（卡号1010357600000007）因银行卡丢失，申请挂失。

```
UPDATE cardinfo  
SET isreportloss = '是'  
WHERE cardid = '1010357600000007';
```

10	1010357600000007	RMB	3	2025-03-03 11:38:24	200.00	200.00	888888	是
----	------------------	-----	---	---------------------	--------	--------	--------	---

图1.5 银行卡挂失

注意事项

- (1) 插入数据时，SQL语法中注意拼写和中英文符号问题。
- (2) 增加新数据时，注意表的约束关系。

随堂测试

1. 查找岗位是工程师且薪水在6000以上的记录，逻辑表达式为（ ）

- A. 岗位 = '工程师' OR 薪水 > 6000
- B. 岗位 = 工程师 AND 薪水 > 6000
- C. 岗位 = 工程师 OR 薪水 > 6000
- D. 岗位 = '工程师' AND 薪水 > 6000

2. WHERE子句中，表达式“age between 20 and 30”等同于（ ）

A. age ≥ 20 and age ≤ 30

B. age ≥ 20 or age ≤ 30

C. age > 20 and age < 30

D. age > 20 and age < 30

3. 从student表中查询学生姓名、年龄和成绩，结果按照年龄降序排序，年龄相同的按照成绩升序排序，下面SQL语句正确的是（ ）

A. select name, age, score from student order by age desc , score;

B. select name, age, score from student order by age , score asc;

C. select name, age, score from student order by 2 desc , 3 asc;

D. select name, age, score from student order by 1 desc , 2 ;

4. 使用SQL命令将STAFFS表中员工的年龄AGE字段的值增加5岁，应该使用的命令是（ ）

A. UPDATE SET AGE WITH AGE+5

B. UPDATE AGE WITH AGE+5

C. UPDATE STAFFS SET AGE = AGE+5

D. UPDATE STAFFS AGE WITH AGE+5

5. 下面四组SQL命令，全部属于数据操作语言的命令是（ ）

A. CREATE, DROP, UPDATE

B. INSERT, UPDATE, DELETE

C. INSERT, DROP, ALTER

D. UPDATE, DELETE, ALTER

6. 删除表student中班级(cid)为6的全部学生信息, 下面SQL语句正确的是 ()

A. delete from student where cid = 6;

B. delete * from student where cid = 6;

C. delete from student on cid = 6;

D. Delete * from student on cid = 6;